🏭 CLAUDE CODE MASTER PROMPT

CRM Digital FTE Factory — Complete Autonomous Build Guide

> **Is prompt ko Claude Code mein paste karo aur poora project khud ban jayega.** Credentials kabhi show nahi honge. No human-in-the-loop required.

📋 DOCUMENT: Project Summary (Pehle yeh parho)

Kya ban raha hai: Ek AI "Digital Employee" (Customer Success FTE) jo 24/7 customer support handle kare — Gmail, WhatsApp, aur Web Form se — bina kisi human ke.

Tech Stack:

- **Agent:** OpenAI Agents SDK (Python)
- **API:** FastAPI
- **Database/CRM:** PostgreSQL + pgvector
- **Events:** Apache Kafka
- **Infra:** Kubernetes (Docker)
- **Frontend:** React/Next.js (Support Form)
- **Channels:** Gmail API, Twilio WhatsApp

🚀 MASTER PROMPT — Claude Code ko yeh dena hai

```
═══════════════════════════════════════════════════════════════
AUTONOMOUS PROJECT BUILD — CRM DIGITAL FTE FACTORY (HACKATHON 5)
═══════════════════════════════════════════════════════════════

You are an expert software architect and full-stack engineer.
Build the COMPLETE "Customer Success Digital FTE" project described below.

⚠️ CRITICAL RULES (NEVER VIOLATE):
1. NEVER hardcode any API keys, passwords, tokens, or credentials in code
2. ALL secrets go in .env file — code only reads from environment variables
3. .env is listed in .gitignore — never committed
4. .env.example has placeholder values only (no real secrets)
5. No human-in-the-loop during build — make ALL decisions autonomously
6. Every UI must be fully mobile responsive (Tailwind CSS)
```

7. Write production-quality code with proper error handling

==

PHASE 1: PROJECT SCAFFOLD (Do this first)

==

Create this EXACT folder structure:

```
customer-success-fte/
├── .env.example              ← Placeholder env vars (no real values)
├── .env                      ← GITIGNORED - real secrets go here
├── .gitignore
├── README.md
├── docker-compose.yml        ← Full local dev environment
├── Dockerfile
├── requirements.txt
│
├── context/
│   ├── company-profile.md    ← Create fake SaaS company "TechFlow"
│   ├── product-docs.md       ← Create 20+ product FAQs
│   ├── sample-tickets.json   ← Create 50+ sample tickets (all 3 channels)
│   ├── escalation-rules.md   ← Define escalation triggers
│   └── brand-voice.md        ← Company tone guidelines
│
├── agent/
│   ├── __init__.py
│   ├── customer_success_agent.py   ← Main OpenAI Agent
│   ├── tools.py                    ← All @function_tool definitions
│   ├── prompts.py                  ← System prompts
│   └── formatters.py               ← Channel-specific formatters
│
├── channels/
│   ├── __init__.py
│   ├── gmail_handler.py            ← Gmail API integration
│   ├── whatsapp_handler.py         ← Twilio WhatsApp integration
│   └── web_form_handler.py         ← Web form API handler
│
├── workers/
│   ├── __init__.py
│   ├── message_processor.py        ← Kafka consumer + agent runner
│   └── metrics_collector.py        ← Background metrics worker
│
├── api/
│   ├── __init__.py
```

```
│   └── main.py                ← FastAPI application
│
├── database/
│   ├── schema.sql             ← Full PostgreSQL schema
│   ├── migrations/
│   │   └── 001_initial.sql
│   └── queries.py             ← Database access layer
│
├── kafka_client.py            ← Kafka producer/consumer
│
├── web-form/                  ← Next.js/React support form
│   ├── package.json
│   ├── next.config.js
│   ├── tailwind.config.js
│   ├── src/
│   │   ├── app/
│   │   │   ├── layout.tsx
│   │   │   ├── page.tsx       ← Main support page
│   │   │   └── globals.css
│   │   └── components/
│   │       ├── SupportForm.tsx     ← REQUIRED: Full form UI
│   │       ├── TicketStatus.tsx    ← Ticket tracking UI
│   │       └── SuccessMessage.tsx  ← Submission success UI
│
├── k8s/
│   ├── namespace.yaml
│   ├── configmap.yaml
│   ├── secrets.yaml
│   ├── deployment-api.yaml
│   ├── deployment-worker.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   └── hpa.yaml
│
├── tests/
│   ├── test_agent.py
│   ├── test_channels.py
│   ├── test_e2e.py
│   └── load_test.py           ← Locust load tests
│
└── specs/
    ├── discovery-log.md
    ├── customer-success-fte-spec.md
    └── transition-checklist.md
```

==========

PHASE 2: ENVIRONMENT & SECRETS SETUP

==========

Create .env.example with THESE EXACT variable names (placeholder values only):

==========

CUSTOMER SUCCESS FTE - Environment Variables

Copy to .env and fill in real values

NEVER commit .env to version control

==========

OpenAI

OPENAI_API_KEY=sk-your-openai-api-key-here
OPENAI_MODEL=gpt-4o

PostgreSQL Database

POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=fte_db
POSTGRES_USER=fte_user
POSTGRES_PASSWORD=your-strong-password-here

Kafka

KAFKA_BOOTSTRAP_SERVERS=localhost:9092

Gmail API (from Google Cloud Console)

GMAIL_CLIENT_ID=your-gmail-client-id.apps.googleusercontent.com GMAIL_CLIENT_SECRET=your-gmail-client-secret GMAIL_REFRESH_TOKEN=your-gmail-refresh-token GMAIL_SENDER_EMAIL=support@yourcompany.com GMAIL_PUBSUB_TOPIC=projects/your-project/topics/gmail-push

Twilio WhatsApp

TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

TWILIO_AUTH_TOKEN=your-twilio-auth-token
TWILIO_WHATSAPP_NUMBER=whatsapp:+14155238886

App Config

APP_ENV=development APP_SECRET_KEY=your-random-secret-key-min-32-chars
CORS_ORIGINS=http://localhost:3000,http://localhost:8000 MAX_EMAIL_LENGTH=2000
MAX_WHATSAPP_LENGTH=1600 MAX_WEBFORM_LENGTH=1000

Redis (optional caching)

REDIS_URL=redis://localhost:6379

Create .gitignore:

.env *.pyc **pycache**/ .pytest_cache/ node_modules/ .next/ *.log venv/ .venv/ dist/ build/ *.egg-info/

═══

PHASE 3: CONTEXT FILES — Create Fake SaaS Company "TechFlow"

═══

context/company-profile.md — Create a realistic SaaS company:
- Name: TechFlow Analytics
- Product: Cloud-based data analytics platform
- Customers: SMBs and startups
- Plans: Starter ($29/mo), Pro ($99/mo), Enterprise (custom)
- Team: 50 employees, 500+ customers

context/product-docs.md — Create 20+ detailed FAQs covering:
1. Account setup and login issues
2. Password reset procedure
3. API authentication (API keys)
4. Dashboard navigation
5. Data import (CSV, JSON, API)
6. Chart/visualization creation
7. Team member invites
8. Billing and subscription info (general only — no prices for escalation)
9. Data export options
10. Webhook configuration
11. Integration with Slack/Zapier
12. Mobile app usage
13. Data retention policies
14. Two-factor authentication
15. Custom domain setup
16. Report scheduling
17. White-labeling options
18. SSO configuration
19. Rate limits and API quotas
20. Data security and GDPR compliance

context/sample-tickets.json — Create 50+ realistic tickets:
- 20 via email (formal, longer messages)
- 15 via WhatsApp (short, casual messages)
- 15 via web form (medium length)
- Include: basic questions, angry customers, pricing asks, bug reports, feature requests
- Each ticket must have: channel, customer_id, message, timestamp, metadata

context/escalation-rules.md — Define these rules:
- Pricing/billing negotiation → escalate

- Refund requests → escalate to billing
- Legal threats (lawyer, sue, lawsuit) → immediate escalation
- Sentiment score < 0.3 (very negative) → escalate
- Customer explicitly asks for human
- WhatsApp: "human", "agent", "representative" → escalate
- Cannot resolve after 2 knowledge base searches → escalate

context/brand-voice.md — Define TechFlow's communication style:
- Friendly but professional
- Empathetic first, solution second
- No jargon, plain language
- Channel-appropriate: formal email, casual WhatsApp

==

PHASE 4: DATABASE SCHEMA (PostgreSQL + pgvector)

==

Build database/schema.sql with these tables (COMPLETE implementation):

1. customers — unified customer table (email as primary identifier)
2. customer_identifiers — cross-channel identity mapping
3. conversations — conversation sessions with channel tracking
4. messages — all messages with inbound/outbound, channel, delivery status
5. tickets — support tickets with lifecycle management
6. knowledge_base — product docs with vector embeddings (pgvector)
7. channel_configs — per-channel settings
8. agent_metrics — performance tracking per channel

Include:
- All foreign keys and constraints
- All indexes for performance (email, phone, status, channel, created_at)
- pgvector index: CREATE INDEX ... USING ivfflat (embedding vector_cosine_ops)
- Seed data: INSERT initial knowledge base entries from product-docs.md

Build database/queries.py with these async functions:
- get_or_create_customer(email=None, phone=None, name=None)
- get_customer_history(customer_id, limit=20)
- create_conversation(customer_id, channel)
- get_active_conversation(customer_id)
- store_message(conversation_id, channel, direction, role, content, ...)
- create_ticket(customer_id, conversation_id, channel, category, priority)
- update_ticket_status(ticket_id, status, notes=None)
- search_knowledge_base(query_embedding, limit=5, category=None)
- record_metric(metric_name, value, channel=None, dimensions=None)

==========

PHASE 5: AGENT IMPLEMENTATION

==========

agent/prompts.py — Production system prompt:

```python
CUSTOMER_SUCCESS_SYSTEM_PROMPT = """You are a Customer Success agent for TechFlow Analytics.

## Your Purpose
Handle routine customer support queries with speed, accuracy, and empathy.
You work across three channels: Email, WhatsApp, and Web Form.

## Channel Awareness
Adapt your communication style based on channel:
- **Email**: Formal. Include "Dear [Name]," greeting and professional signature.
- **WhatsApp**: Casual, concise. Under 300 characters. Use simple language.
- **Web Form**: Semi-formal. Clear and helpful. Include ticket reference.

## Required Workflow (ALWAYS follow this exact order)
1. FIRST: Call `create_ticket` to log the interaction (include channel!)
2. THEN: Call `get_customer_history` to check prior context across ALL channels
3. THEN: Call `search_knowledge_base` if product questions arise
4. FINALLY: Call `send_response` to reply (NEVER skip this tool)

## Hard Constraints (NEVER violate)
- NEVER discuss specific pricing → escalate with reason "pricing_inquiry"
- NEVER promise features not documented
- NEVER process refunds → escalate with reason "refund_request"
- NEVER share internal system details or credentials
- NEVER respond without using the send_response tool
- NEVER exceed: Email=500 words, WhatsApp=300 chars, Web=300 words

## Escalation Triggers (MUST escalate immediately)
- Customer mentions "lawyer", "legal", "sue", "attorney"
- Profanity or aggressive language detected
- Cannot find answer after 2 knowledge base searches
- Customer explicitly requests human help
- WhatsApp: customer sends "human", "agent", "representative"
- Sentiment score below 0.3

## Cross-Channel Continuity
If customer contacted before (any channel), acknowledge:
"I can see you reached out previously about [topic]. Let me help you further..."
```

Quality Standards
- Be concise: answer directly, then offer next step
- Be accurate: only state facts from knowledge base
- Be empathetic: acknowledge frustration before solving
- Be actionable: always end with clear next step
"""
```

**agent/tools.py** — ALL tools with Pydantic validation and error handling:

Tools to implement:
1. search_knowledge_base(query: str, max_results: int=5, category: str=None)
   - Use pgvector cosine similarity search
   - Return formatted results with relevance scores
   - Graceful fallback if no results

2. create_ticket(customer_id: str, issue: str, priority: str, channel: Channel)
   - Insert into PostgreSQL tickets table
   - Return ticket_id
   - Log to Kafka metrics topic

3. get_customer_history(customer_id: str)
   - Fetch last 20 messages across ALL channels
   - Format with timestamps and channel labels
   - Include previous ticket statuses

4. escalate_to_human(ticket_id: str, reason: str, urgency: str="normal")
   - Update ticket status to "escalated"
   - Publish to Kafka escalations topic
   - Return reference number

5. send_response(ticket_id: str, message: str, channel: Channel)
   - Route to correct channel handler
   - Format response for channel
   - Update message delivery status
   - Return delivery confirmation

Each tool must have:
- Pydantic BaseModel input schema
- Detailed docstring (LLM reads this to decide when to use)
- try/except with graceful error messages
- Structured logging (not print statements)
```

agent/formatters.py — Channel formatters:
- format_email_response(message, customer_name, ticket_id) → formal email with greeting/signature
- format_whatsapp_response(message) → truncate to 300 chars, add "📱 Reply for more help"
- format_web_response(message, ticket_id) → semi-formal with ticket reference

agent/customer_success_agent.py — Agent definition:
```python
from agents import Agent, Runner, function_tool
# Create agent with all 5 tools
# Model: gpt-4o
# Include full system prompt from prompts.py
```

==========

PHASE 6: CHANNEL INTEGRATIONS

==========

channels/gmail_handler.py — Complete implementation:
- GmailHandler class
- __init__ reads credentials from env vars (NEVER hardcoded)
- setup_push_notifications(topic_name)
- process_notification(pubsub_message) → list of normalized messages
- get_message(message_id) → normalized dict with channel="email"
- send_reply(to_email, subject, body, thread_id=None)
- _extract_body(payload) for multipart emails
- Proper OAuth2 token refresh handling

channels/whatsapp_handler.py — Complete implementation:
- WhatsAppHandler class
- __init__ reads from env vars (TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, etc.)
- validate_webhook(request) → verify Twilio signature
- process_webhook(form_data) → normalized dict with channel="whatsapp"
- send_message(to_phone, body) → delivery status
- format_response(response, max_length=1600) → split long messages

channels/web_form_handler.py — Complete FastAPI router:
- SupportFormSubmission Pydantic model with validators
- POST /support/submit → create ticket, publish to Kafka, return ticket_id
- GET /support/ticket/{ticket_id} → status + conversation history
- GET /support/ticket/{ticket_id}/messages → all messages in conversation

==========

PHASE 7: WEB SUPPORT FORM (REQUIRED — Mobile Responsive)

==========

Build a complete Next.js 14 application in web-form/ directory.

DESIGN REQUIREMENTS:
- Clean, professional design using Tailwind CSS
- Fully mobile responsive (works on 320px screens)
- Fast loading (no unnecessary dependencies)
- Accessible (proper labels, ARIA attributes)
- Real-time character counter for message field
- Smooth loading states with spinners
- Success/error states with clear messaging

Components to build:

SupportForm.tsx — Main form with these fields:
1. Full Name (required, min 2 chars)
2. Email Address (required, valid email format)
3. Subject (required, min 5 chars)
4. Category dropdown: General, Technical Support, Billing, Bug Report, Feedback
5. Priority dropdown: Low, Medium (default), High, Urgent
6. Message textarea (required, min 10 chars, max 1000, with live counter)
7. File attachment (optional, max 5MB, accept: .png,.jpg,.pdf,.txt)
8. Submit button with loading spinner
9. Privacy policy link at bottom

TicketStatus.tsx — After submission:
- Large success checkmark (green)
- "Your request has been submitted!" heading
- Ticket ID displayed in monospace font (copyable)
- Estimated response time: "Usually within 5 minutes"
- "Track your ticket" button → opens status page
- "Submit another request" button

Ticket tracking page — /ticket/[id]:
- Real-time ticket status (open/processing/resolved/escalated)
- Full conversation thread display
- Agent responses highlighted differently from customer messages
- Channel badge (Email/WhatsApp/Web)
- Timestamps for each message

Mobile responsive rules:
- Stack all form columns on mobile (< 640px)
- Full-width inputs on all screens
- Minimum touch target 44px for buttons

- No horizontal scroll on any screen size
- Proper font sizes (min 16px for inputs to prevent iOS zoom)

==
PHASE 8: KAFKA EVENT STREAMING
==

kafka_client.py — Complete implementation:

Topics to create:
- fte.tickets.incoming   ← All incoming from all channels
- fte.channels.email.inbound
- fte.channels.whatsapp.inbound
- fte.channels.webform.inbound
- fte.channels.email.outbound
- fte.channels.whatsapp.outbound
- fte.escalations   ← Human handoff events
- fte.metrics   ← Performance data
- fte.dlq   ← Dead letter queue for failures

Classes:
- FTEKafkaProducer: start(), stop(), publish(topic, event)
- FTEKafkaConsumer: start(), stop(), consume(handler)

==
PHASE 9: FASTAPI APPLICATION
==

api/main.py — Complete FastAPI app:

Endpoints:
- GET /health   ← Liveness check with channel status
- POST /webhooks/gmail   ← Gmail Pub/Sub notifications
- POST /webhooks/whatsapp   ← Twilio webhook (validate signature!)
- POST /webhooks/whatsapp/status ← Delivery status updates
- POST /support/submit   ← Web form (from web_form_handler router)
- GET /support/ticket/{id} ← Ticket status
- GET /conversations/{id} ← Full conversation history
- GET /customers/lookup   ← Find customer by email or phone
- GET /metrics/channels   ← Channel performance metrics
- GET /metrics/daily   ← Daily summary report
- GET /docs   ← Auto-generated Swagger UI

Middleware:

- CORS (origins from env var CORS_ORIGINS)
- Request logging with correlation IDs
- Error handler that never exposes internal details

==
PHASE 10: WORKERS
==

workers/message_processor.py — Main processing loop:
- UnifiedMessageProcessor class
- start() → consume from fte.tickets.incoming
- process_message(topic, message):
  1. Detect channel (email/whatsapp/web_form)
  2. resolve_customer() → get or create customer record
  3. get_or_create_conversation() → link to active session
  4. store incoming message in DB
  5. Load conversation history (last 10 messages)
  6. Run customer_success_agent with context
  7. Store agent response in DB
  8. Publish metrics to Kafka
  9. Log processing time
- handle_error() → send apology + publish to DLQ + notify human

workers/metrics_collector.py — Background metrics:
- Collect hourly stats per channel
- Calculate: avg response time, escalation rate, resolution rate
- Store in agent_metrics table
- Generate daily summary report

==
PHASE 11: DOCKER & LOCAL DEV
==

docker-compose.yml — Complete local environment:

Services:
1. postgres:16
   - Image: pgvector/pgvector:pg16
   - Port: 5432
   - Volume: postgres_data
   - Env from .env file
   - Health check: pg_isready

2. kafka

- Image: confluentinc/cp-kafka:latest
- Port: 9092
- ZooKeeper embedded (KRaft mode)
- Auto-create topics enabled

3. redis:7-alpine
- Port: 6379
- For caching/rate limiting

4. fte-api
- Build from Dockerfile
- Command: uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
- Port: 8000
- Depends on: postgres, kafka
- Env from .env file
- Volume: .:/app for hot reload

5. fte-worker
- Build from Dockerfile
- Command: python workers/message_processor.py
- Depends on: postgres, kafka, fte-api

6. web-form (optional)
- Image: node:20-alpine
- Command: npm run dev
- Port: 3000
- Working dir: /app/web-form

Dockerfile — Multi-stage build:
```dockerfile
FROM python:3.11-slim as base
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# Never copy .env into image!
```

requirements.txt — All dependencies:

```
openai>=1.0.0
openai-agents>=0.0.1
fastapi>=0.104.0
```

uvicorn[standard]>=0.24.0
asyncpg>=0.29.0
aiokafka>=0.10.0
google-auth>=2.23.0
google-api-python-client>=2.100.0
twilio>=8.10.0
pydantic[email]>=2.5.0
pgvector>=0.2.4
python-dotenv>=1.0.0
structlog>=23.2.0
pytest>=7.4.0
pytest-asyncio>=0.23.0
httpx>=0.25.0
locust>=2.19.0

PHASE 12: KUBERNETES MANIFESTS

k8s/ directory — Complete manifests:

1. namespace.yaml — customer-success-fte namespace
2. configmap.yaml — Non-secret config (Kafka URL, response limits, feature flags)
3. secrets.yaml — Template only (values injected by CI/CD, not stored in repo)
4. deployment-api.yaml:
   - 3 replicas
   - Resource limits: 512Mi RAM, 500m CPU
   - Liveness + readiness probes on /health
   - Rolling update strategy
5. deployment-worker.yaml:
   - 3 replicas
   - Auto-scale based on Kafka consumer lag
6. service.yaml — ClusterIP for internal, LoadBalancer for API
7. ingress.yaml — NGINX ingress with TLS
8. hpa.yaml — HPA for both API and Worker (min:3, max:20/30, CPU 70%)

PHASE 13: TESTS

tests/test_agent.py — Unit tests:
- test_pricing_always_escalates()
- test_legal_threat_escalates()
- test_tool_call_order() — create_ticket must be FIRST
- test_email_has_greeting()
- test_whatsapp_is_short() — < 500 chars
- test_empty_message_handled()
- test_cross_channel_history()

tests/test_channels.py — Channel tests:
- test_gmail_message_parsing()
- test_gmail_reply_format()
- test_whatsapp_webhook_validation()
- test_whatsapp_message_splitting()
- test_web_form_validation() — test all validators
- test_web_form_submission()

tests/test_e2e.py — End-to-end tests:
- test_full_web_form_flow()
- test_ticket_status_retrieval()
- test_customer_history_across_channels()
- test_escalation_flow()
- test_metrics_endpoint()

tests/load_test.py — Locust scenarios:
- WebFormUser: submit forms (weight=3)
- HealthCheckUser: monitor health (weight=1)
- Target: 100 concurrent users, < 3s P95 latency

==
PHASE 14: DOCUMENTATION
==

README.md — Complete setup guide:
1. Prerequisites (Python 3.11+, Node 20+, Docker, kubectl)
2. Quick Start with Docker Compose
3. Manual Setup (step by step)
4. Environment Variables guide (reference .env.example)
5. Channel Setup Guides:
   - Gmail API setup (Google Cloud Console steps)
   - Twilio WhatsApp Sandbox setup
6. Running Tests
7. Kubernetes Deployment
8. API Documentation link
9. Architecture diagram (ASCII)
10. Troubleshooting common issues

specs/discovery-log.md — Document incubation discoveries:
- Channel-specific patterns found
- Edge cases discovered
- Response format decisions made
- Escalation rule refinements

specs/customer-success-fte-spec.md — Full specification:
- Supported channels table
- In-scope / out-of-scope features
- Tools manifest
- Performance requirements
- Guardrails

==

PHASE 15: FINAL CHECKS

After building everything, verify:

✅ SECURITY CHECKLIST:
- [ ] No API keys in any .py, .ts, .json, .yaml, .md file
- [ ] .env is in .gitignore
- [ ] Docker image does not COPY .env
- [ ] K8s secrets.yaml has no real values
- [ ] All credentials read via os.getenv() or python-dotenv

✅ FUNCTIONALITY CHECKLIST:
- [ ] docker-compose up starts all services cleanly
- [ ] GET /health returns 200
- [ ] POST /support/submit creates ticket and returns ticket_id
- [ ] Web form renders correctly on mobile (375px width)
- [ ] All 5 agent tools have Pydantic validation
- [ ] Error handling in all tools (no raw exceptions to user)
- [ ] Kafka topics defined for all 3 channels

✅ MOBILE RESPONSIVENESS:
- [ ] Support form works on 320px screen
- [ ] No horizontal scroll on any page
- [ ] Touch targets minimum 44x44px
- [ ] Input font size minimum 16px (prevents iOS zoom)
- [ ] Ticket status page readable on mobile

✅ CODE QUALITY:
- [ ] All async functions properly awaited
- [ ] Connection pools reused (not created per request)
- [ ] Structured logging (structlog) not print()
- [ ] All database queries use parameterized inputs (no SQL injection)

START BUILDING NOW

Begin with Phase 1 (folder structure), then proceed in order.
Make all architectural decisions autonomously.
If a dependency version is unclear, use the latest stable version.
Document any assumptions in README.md.

```
Build the COMPLETE project. Do not stop at examples or placeholders.
Every file must be fully implemented and functional.
```

📁 Yeh Prompt Kaise Use Karein

Step 1 — Claude Code Install Karo

```bash
npm install -g @anthropic-ai/claude-code
cd ~/Desktop
mkdir my-fte-project && cd my-fte-project
claude
```

Step 2 — Prompt Paste Karo

Upar wala poora prompt copy karo aur Claude Code mein paste karo. Enter dabao.

Step 3 — Credentials Baad Mein Bhar Do

Jab project ban jaye:

```bash
cp .env.example .env
# .env file open karo aur apni real credentials dalo
nano .env
```

Step 4 — Run Karo

```bash
docker-compose up --build
# API: http://localhost:8000
# Web Form: http://localhost:3000
# API Docs: http://localhost:8000/docs
```

🔐 Credentials Guide — Kahan Se Milenge

Variable	Kahan Se Milega
`OPENAI_API_KEY`	platform.openai.com → API Keys
`GMAIL_*` credentials	console.cloud.google.com → Gmail API
`TWILIO_*` credentials	console.twilio.com → Account Info
`POSTGRES_PASSWORD`	Apna khud banana (strong random string)
`APP_SECRET_KEY`	`python -c "import secrets; print(secrets.token_hex(32))"`

⚡ Quick Tips

1. **Pehle local test karo** — Docker Compose se, Kubernetes baad mein
2. **WhatsApp ke liye** — Twilio Sandbox use karo (free, no approval needed)
3. **Gmail ke liye** — Google Cloud Console mein test account use karo
4. **pgvector** — PostgreSQL extension install hona zaruri hai (docker image mein already hai)
5. **Kafka UI** — `confluentinc/cp-enterprise-control-center` add kar sakte ho docker-compose mein debugging ke liye

---

Yeh prompt Claude Code ko complete autonomous build ke liye design kiya gaya hai. Koi bhi credential code mein nahi ayega. Sab kuch .env mein rahega jo gitignore mein listed hai.